HEP Agent Simulation Framework

Interface Documentation

June 22, 2026

Contents

1 Introduction **3**

2 Main Graphical Interface **4**

3 Configuration Tab **6**
3.1 Configuration List 6
3.2 HEP Input Selection 8

4 Spawn Editor Tab **10**
4.1 Module Configuration 11
4.1.1 Test Modules 12
4.1.2 Yaping Development Modules 12
4.2 Spawn Point Management 12
4.3 Preview Context 13
4.4 Initial Age Distribution 14

5 View Editor Tab **16**
5.1 Map View Settings 16
5.2 Clustering Algorithms 17
5.3 Plot Configuration 17

6 Full Simulation Tab **18**
6.1 Parameters 18

7 Debugging Fortran Modules **19**
7.1 Development Workflow Overview 19
7.2 Print-Based Debugging 20
7.3 Common Crash Causes 20
7.4 Configuration Validation 21
7.5 Debug Counters Overlay 21
7.6 Rebuild & Test Cycle 22

8 Python–Fortran Execution Flow 23
8.1 Initialization Sequence 23
8.1.1 Interactive Mode (GUI) 24
8.1.2 Headless Mode 24
8.2 Simulation Step Execution 24
8.2.1 Interactive Mode (GUI) Loop 25
8.2.2 Headless Mode Loop 25

9 Recent Changes 26

10 Project State Summary - May 5, 2026 27
10.1 Objective 27
10.2 Completed Work 27
10.2.1 1. Dead Agent Archive & NetCDF Storage (April 21st) 27
10.2.2 2. New Module Pattern & World Data Access (April 23rd) 28
10.2.3 3. DBSCAN, Convex Hull & Performance Metrics (April 28th) 28
10.2.4 4. Unified Density Smoothing & Global Metrics (April 29th) 28
10.2.5 5. Birth-Death Refactoring & Logic Consolidation (May 4th) 29
10.2.6 6. Architectural Refinement & Critical Fixes (April - May) 29
10.3 TODO later 30
10.3.1 7. Cluster Population Control & Multi-Population Graphing (May 13th) 30
10.3.2 8. Performance Metrics HUD & Population-Specific Mating (May 26th) 31
10.3.3 9. Critical Bugfix: Hashmap Population Corruption (May 26th) 32
10.3.4 10. Efficient Cluster-Restricted Density Updates (May 26th) 32
10.3.5 11. Responsive Rolling Average for Performance HUD (May 26th) 34
10.3.6 12. HEP Data Chunked Loading (May 26th) 34
10.3.7 13. Clustering Fix: "Clusters Way Too Small" (May 26th) 35
10.3.8 14. Bugfix: Out-of-Bounds Array Access in Cluster Accumulators (May 26th) 36
10.4 Tripwires & Lessons Learned 36

Chapter 1

Introduction

The HEP Agent Simulation Framework provides a graphical user interface (GUI) to configure the fortran simulation.

Chapter 2

Main Graphical Interface

When launching ‘application.py‘, the user is presented with the main window containing four primary tabs (see Figure 2.2):

- **Configuration**: For managing simulation settings and HEP inputs.
- **Spawn Editor**: For defining initial agent distribution.
- **View Editor**: For configuring visualization settings.
- **Full Simulation**: For running headless, long-duration simulations.

At the bottom of the window, buttons are available to launch the simulation (Figure 2.1).

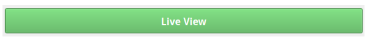

Figure 2.1: Live View Button

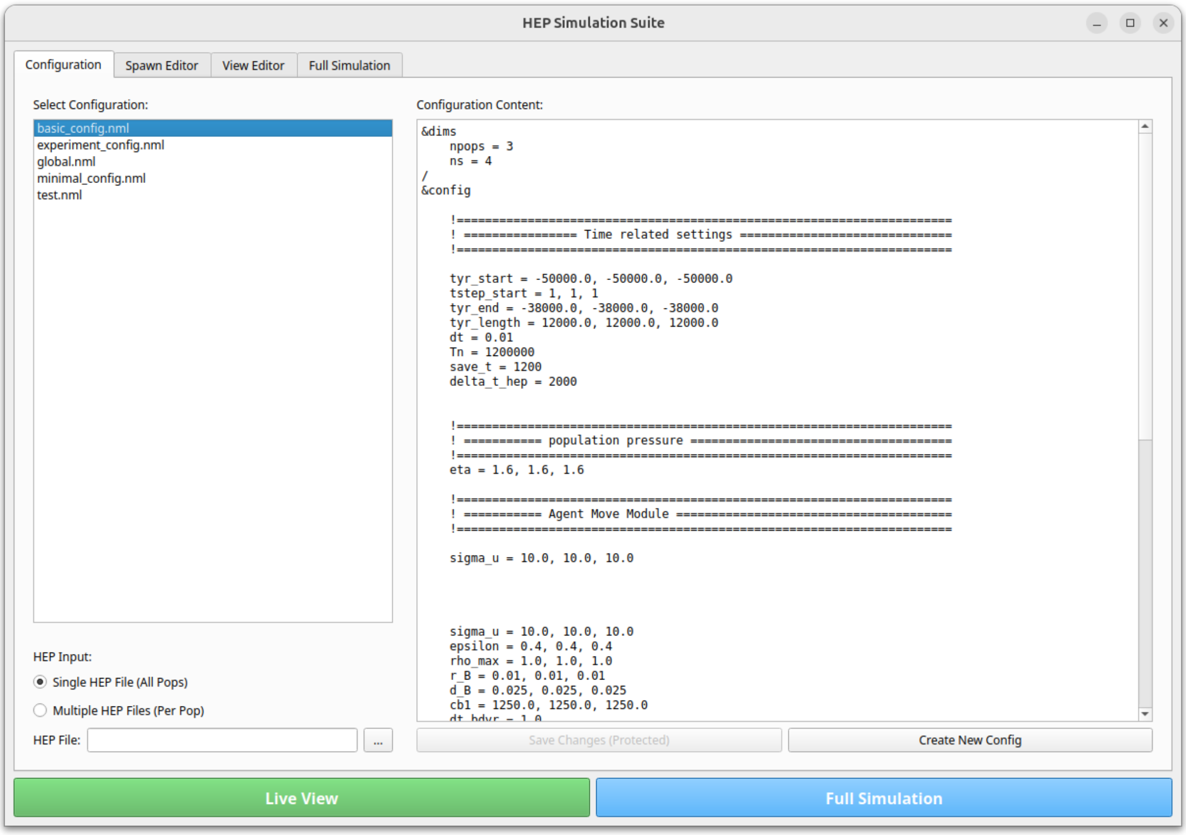

Figure 2.2: Overview of the Main Application Window (Configuration Tab)

Chapter 3

Configuration Tab

The Configuration tab serves as the entry point for setting up the simulation parameters.

3.1 Configuration List

On the left side, a list of available ‘.nml‘ configuration files is displayed (see Figure 3.1). Users can select a configuration to view and edit its content in the right-hand text editor.

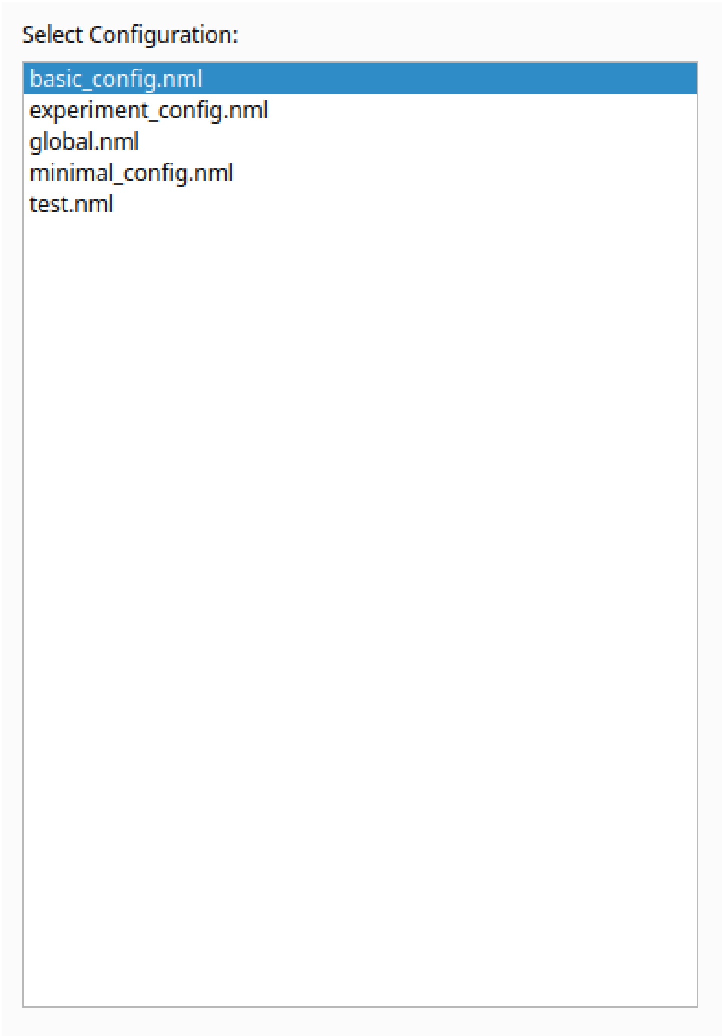

Figure 3.1: Selecting a Configuration File

The editor allows direct modification of the simulation parameters within the application (Figure 3.2). Note that the `basic_config.nml` serves as a template and cannot be modified directly in the editor.

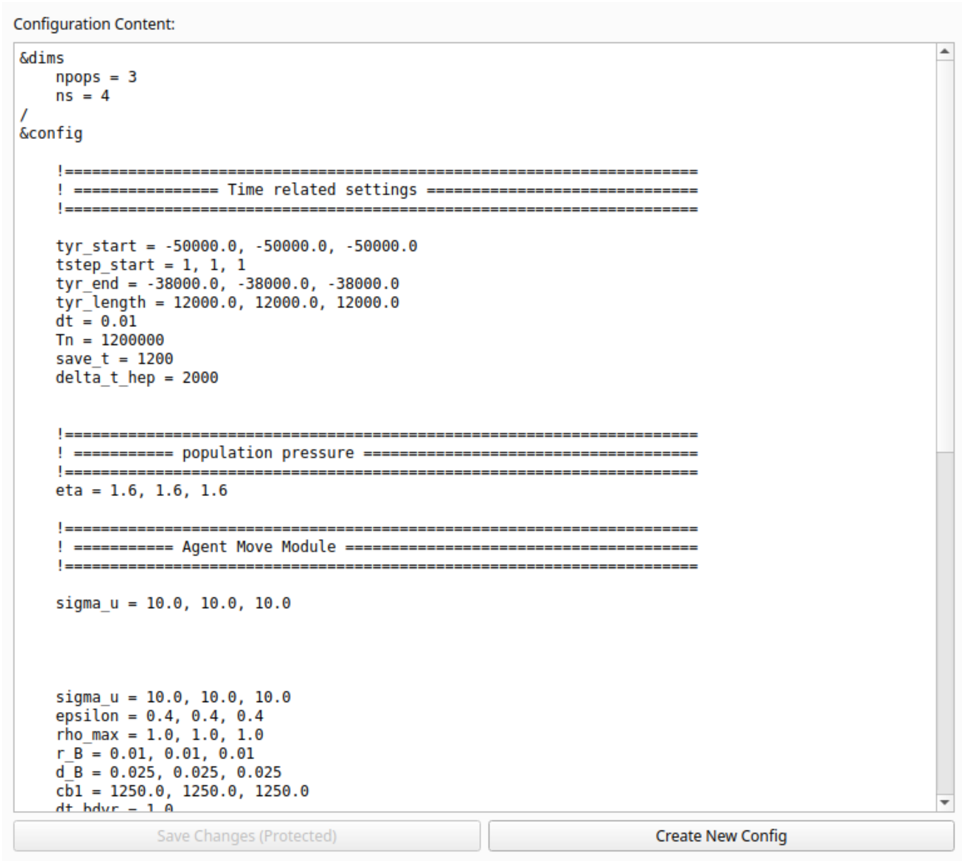

Figure 3.2: Editing Configuration Parameters

3.2 HEP Input Selection

Users can select the HEP input files (Figure 3.3):

- **Single HEP File**: Uses the same hep file for all populations.
- **Multiple HEP Files**: Allows specifying distinct hep files for each population.

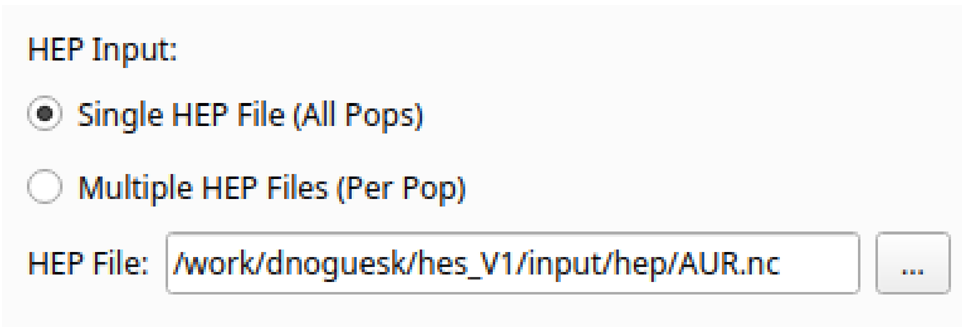

Figure 3.3: Selecting HEP Input Files

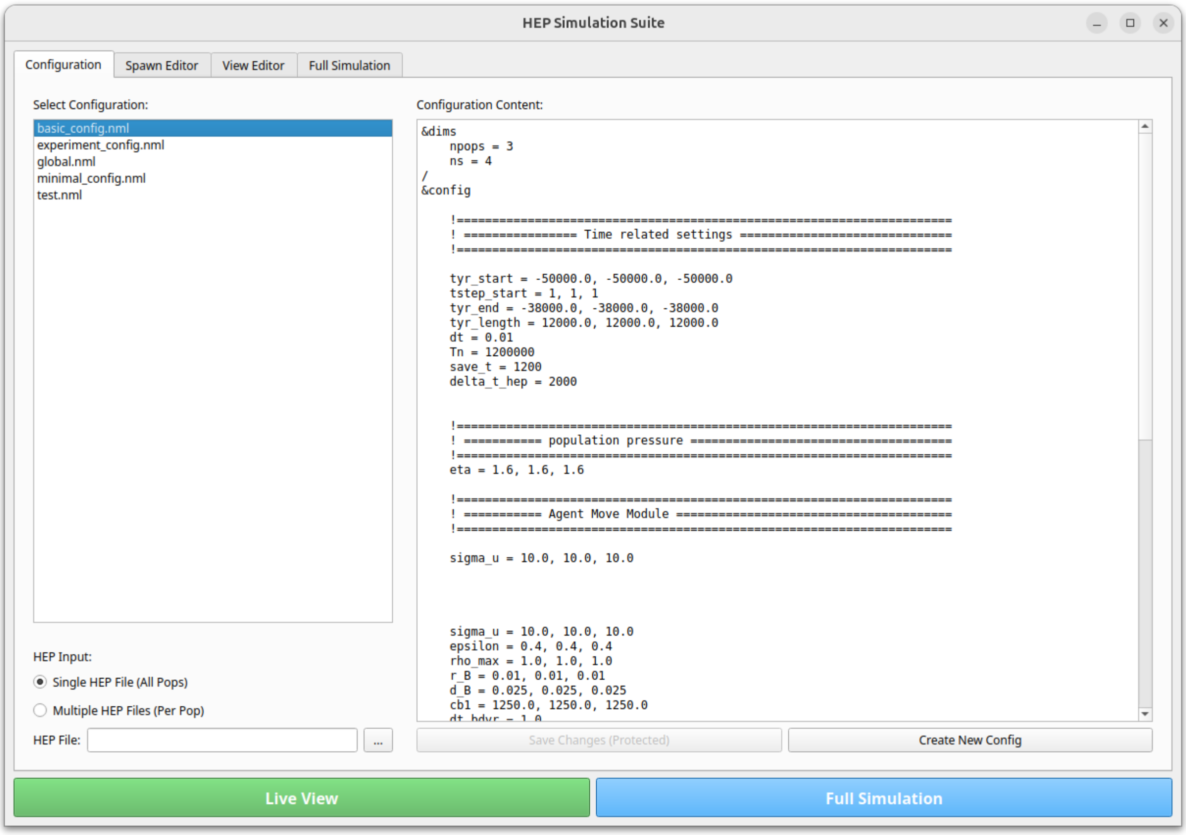

Figure 3.4: Configuration Tab showing file selection and editor

Chapter 4

Spawn Editor Tab

The Spawn Editor allows users to define where agents are initially placed on the map, configure active simulation modules, and set initial age distributions (Figure 4.1). This tab is only available after a HEP file and a configuration file have been selected. The left panel is scrollable to accommodate all controls on smaller screens.

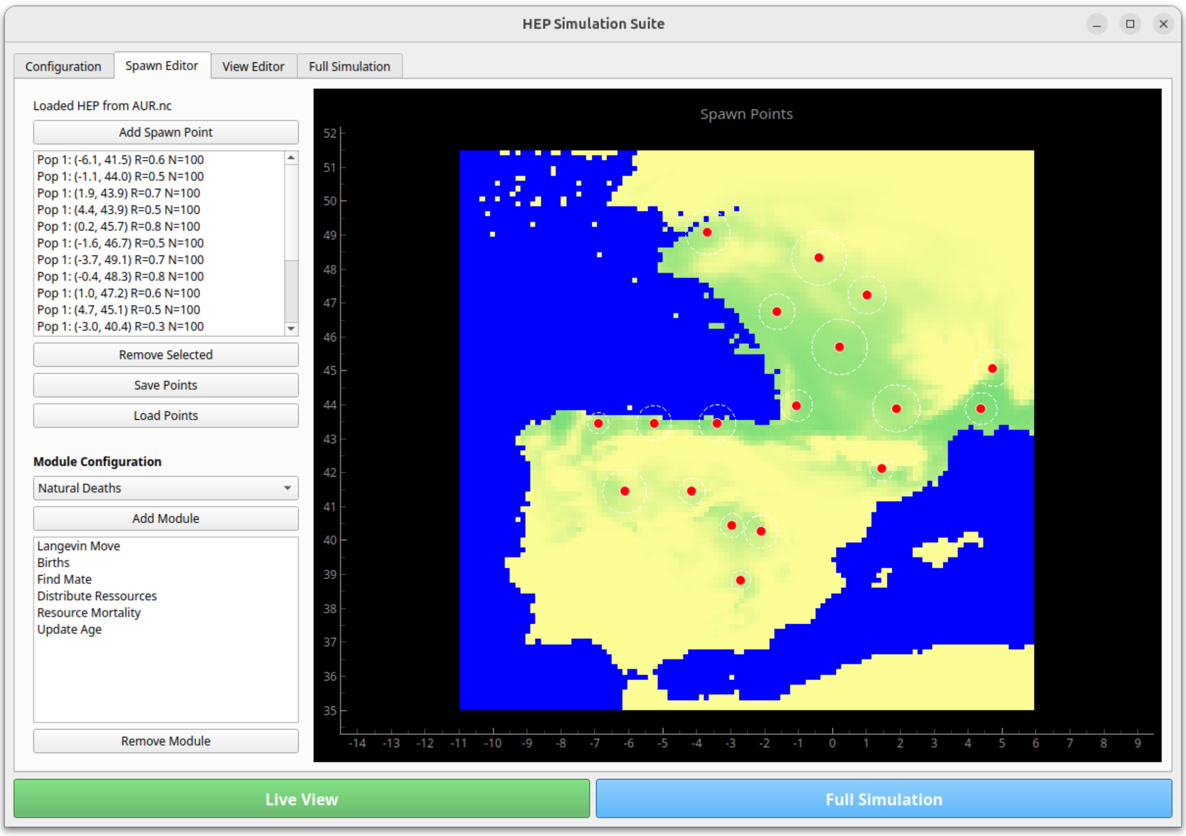

Figure 4.1: Spawn Editor Interface

4.1 Module Configuration

The user can activate or deactivate specific simulation modules for the agent population (see Figure 4.2). Modules are displayed in a **grouped tree view** with three columns:

Module The name of the module.

Author Who developed the module.

File The Fortran source file where the module is implemented.

Modules are organised into groups such as *Core*, *ProofOfConcept*, *Movement*, *Templates*, and *YapingDevelopment*. To add a module, select it in the tree and click "Add Module" (or double-click it). Active modules appear in the list below and can be reordered by drag-and-drop.

Note that new modules only appear in this tree after the Fortran code has been recompiled and registered in the framework.

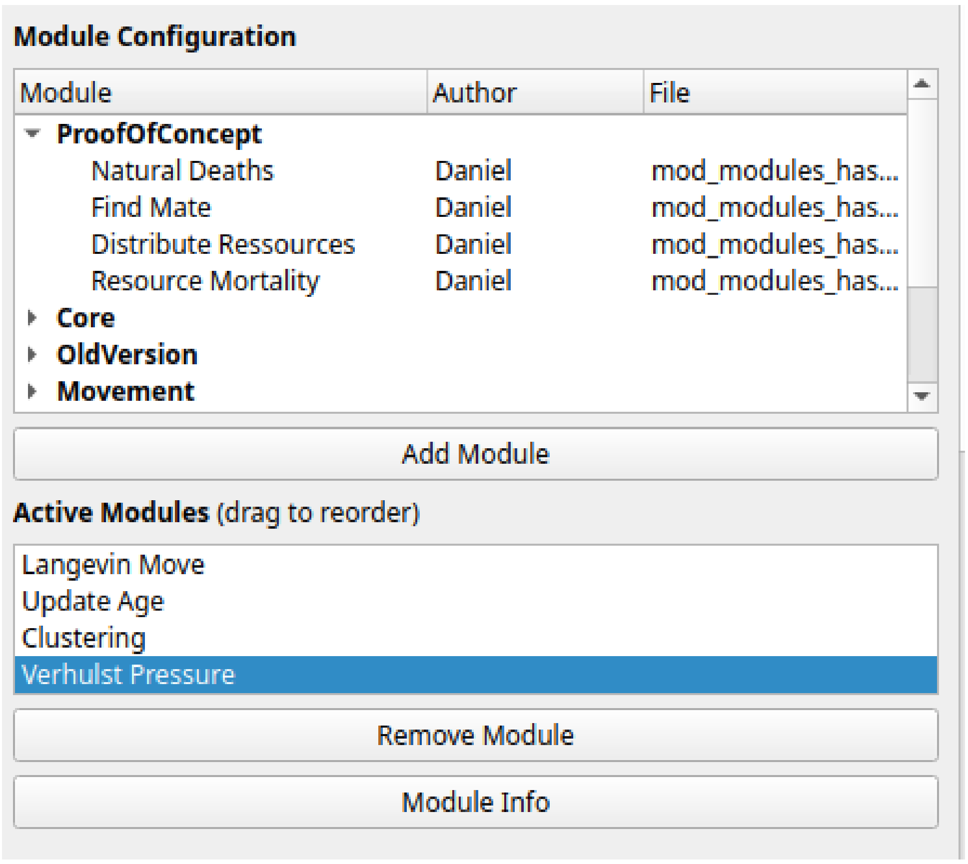

Figure 4.2: Module Configuration Panel showing grouped modules with Module, Author, and File columns.

4.1.1 Test Modules

Two special template modules are included for rapid prototyping:

Test Module (Agents) Runs user-defined code once per tick on every living agent. Use this when your logic operates on individual agents (e.g., movement rules, death conditions).

Test Module (Grid) Runs user-defined code once per tick on the entire spatial grid. Use this when your logic depends on spatial relationships between agents or grid-cell properties.

To use them, edit `src/simulation_modules/mod_test_modules.f95`, write your logic in the clearly marked sections, rebuild with `./build.sh`, and toggle the module on in this panel. For a detailed walkthrough, see Chapter 3 of the *Code Documentation*.

4.1.2 Yaping Development Modules

Four development modules are available for Prof. Yaping's research group, all implemented in `mod_yaping_development.f95`:

Yaping Move Agent-centric movement module.

Yaping Birth Grid Grid-centric birth module.

Yaping Death AGB Agent-centric death module (age/gender-based).

Yaping Death Grid Grid-centric death module.

These follow the same template pattern as the test modules: edit the source file, rebuild, and activate in the module panel.

4.2 Spawn Point Management

Users can add, remove, and manage spawn points directly from the list (Figure 4.3).

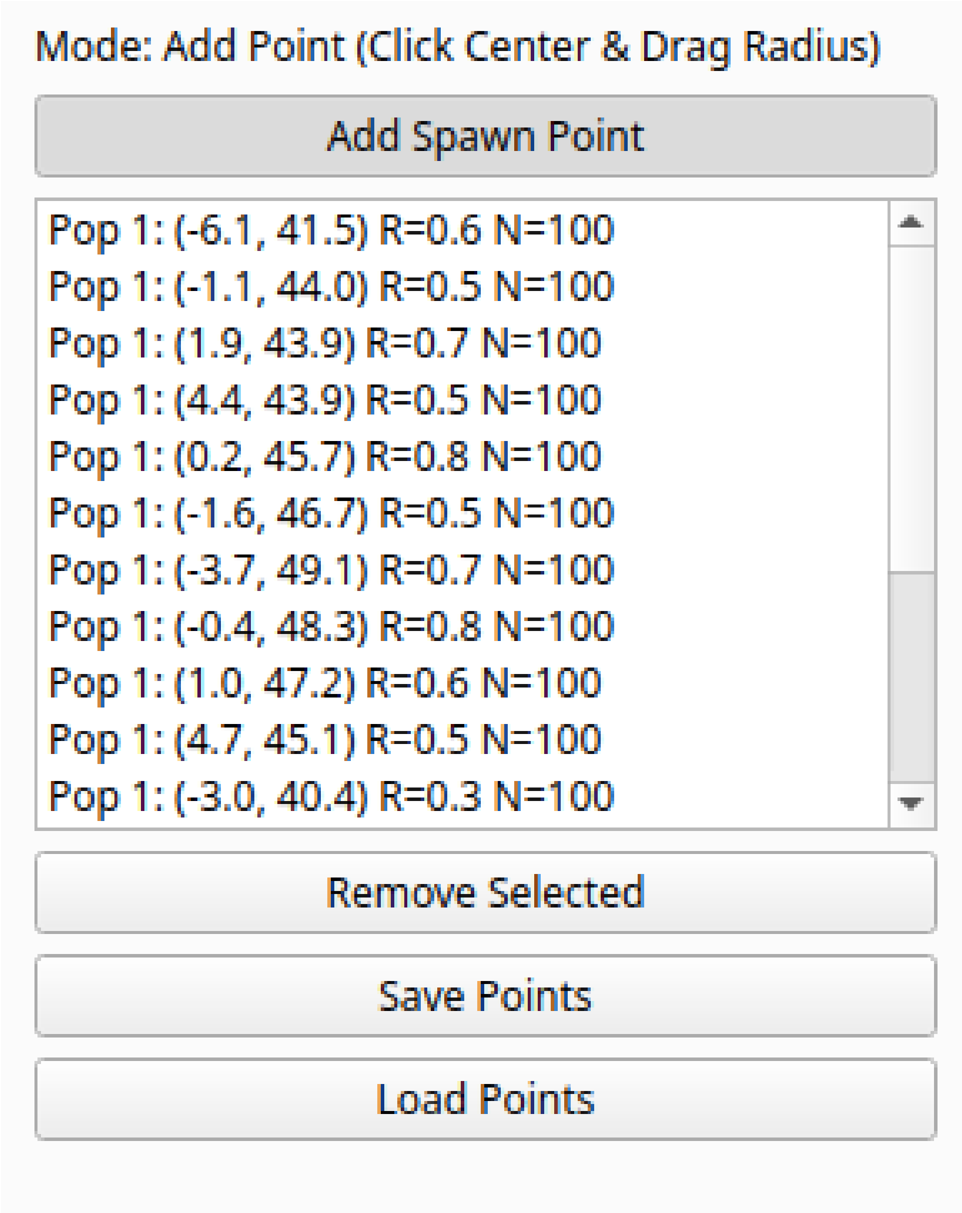

Figure 4.3: Spawn Point Management List

4.3 Preview Context

The editor allows previewing spawn locations overlaid on the selected HEP environment (Figure 4.4).

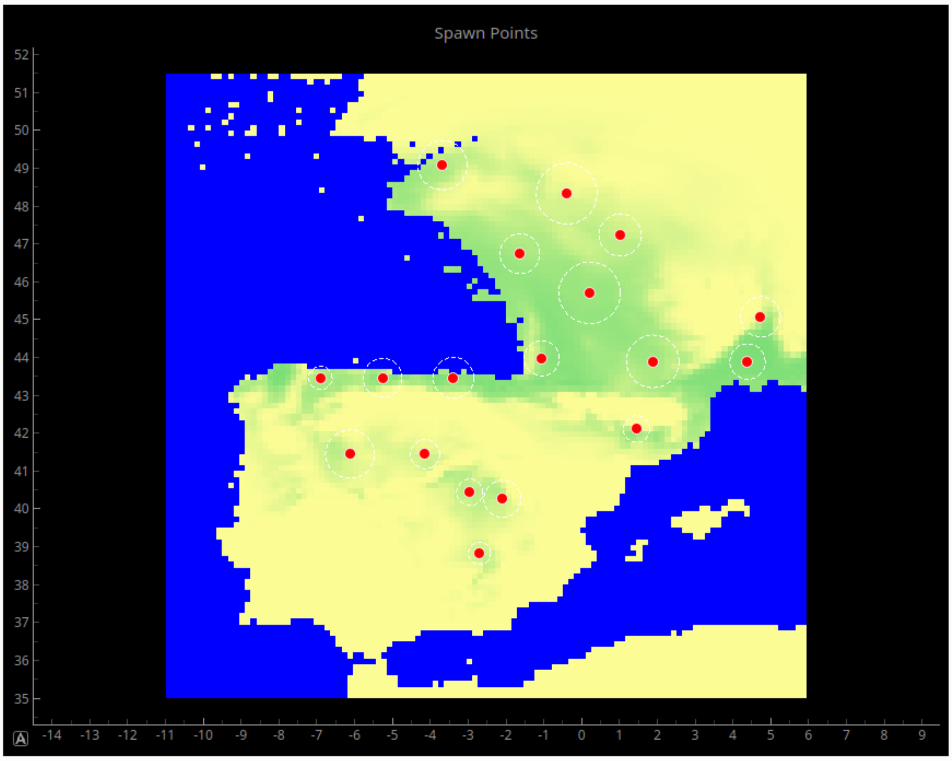

Figure 4.4: Preview of Spawn Points on HEP Map

4.4 Initial Age Distribution

The Spawn Editor includes an optional age distribution feature (Figure 4.5). When enabled, agents are spawned with ages drawn from a configured distribution rather than all starting at age zero.

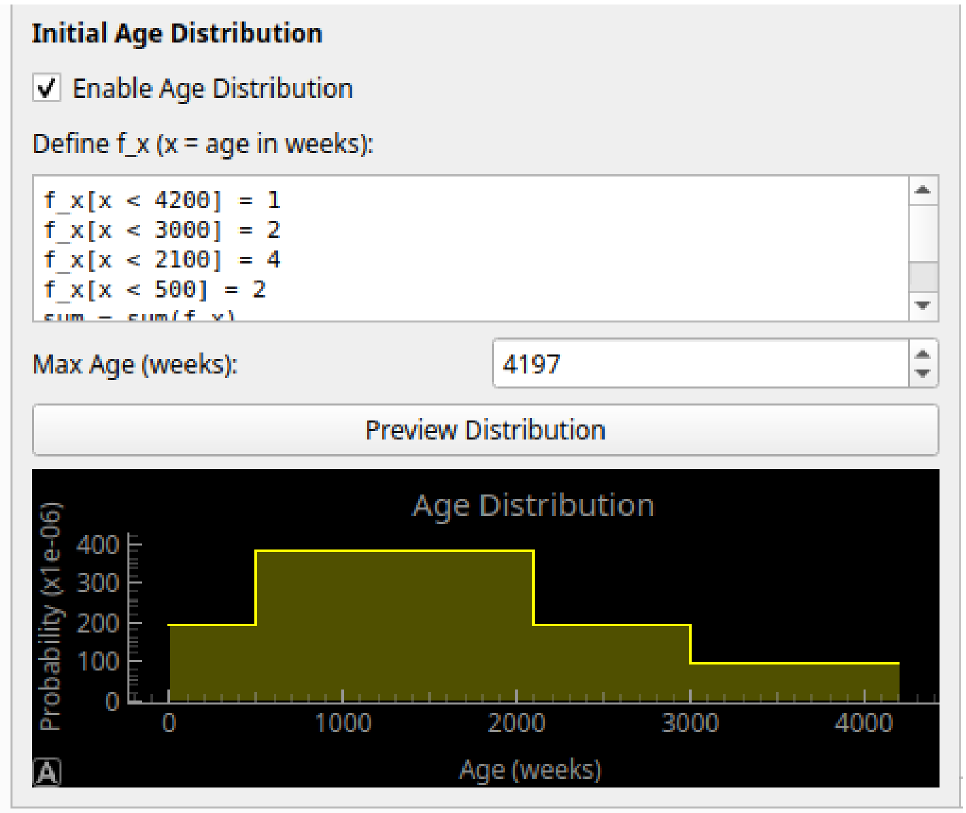

Figure 4.5: Initial Age Distribution configuration and preview.

Users can:

- Enable or disable the age distribution with a checkbox.
- Configure the distribution parameters.
- Preview the resulting distribution as a histogram before starting the simulation.

Chapter 5

View Editor Tab

The View Editor controls the visualization parameters for the Live View execution (see Figure 5.1).

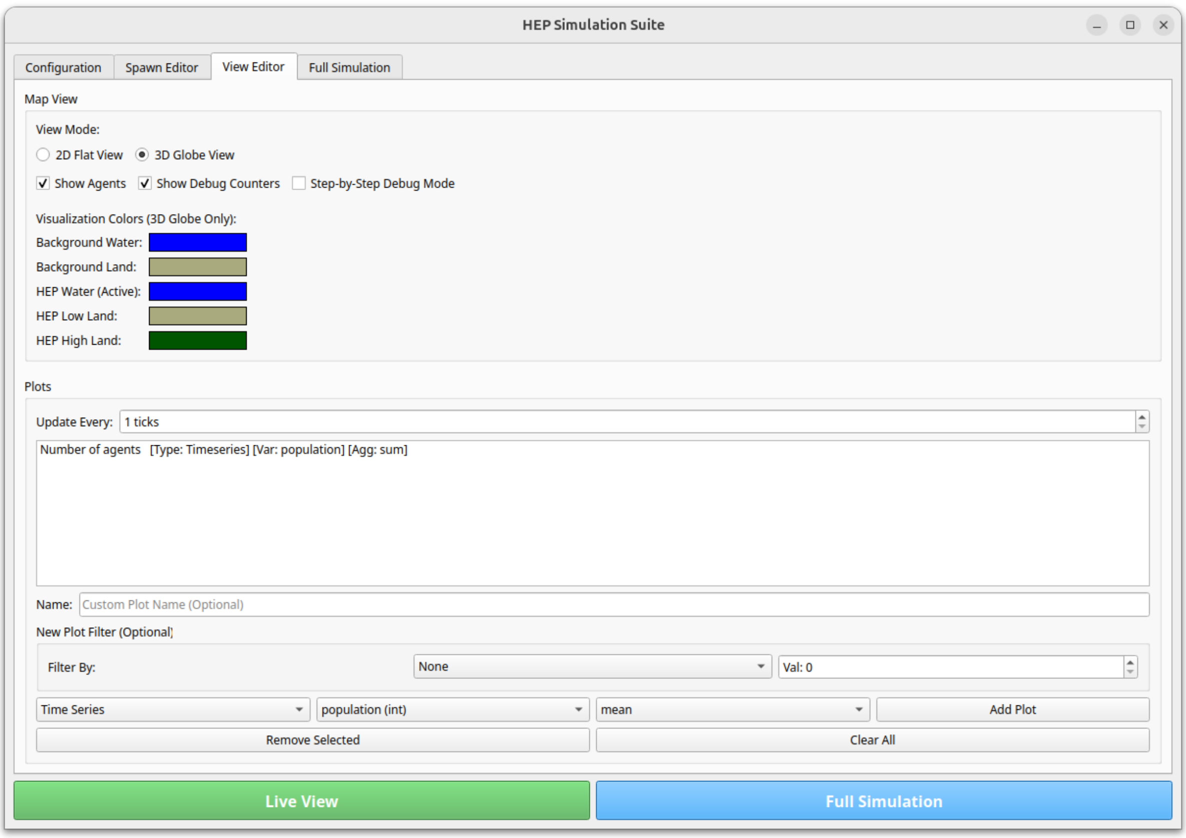

Figure 5.1: View Editor Overview

5.1 Map View Settings

Users can toggle between visualization modes:

- **2D Flat View**: Standard map projection.
- **3D Globe View**: Spherical projection of the simulation data.

5.2 Clustering Algorithms

The visualization engine supports several algorithms for real-time population analysis:

- **Watershed**: Uses gradient ascent on the smoothed density surface.
- **K-Means**: Classical centroid-based clustering.
- **DBSCAN**: Density-based clustering for identifying arbitrary shapes and noise. Controls for **eps** and **minPts** are available in the UI.
- **Auto K-Means**: Automatically detects the number of clusters by finding local density maxima.

5.3 Plot Configuration

A dynamic plotting interface allows users to add real-time analysis charts. Supported plot types include:

- **Time Series**: Tracks variable evolution over time (e.g., Total Population).
- **Bucket Plot**: Demographics distribution (e.g., Age cohorts).
- **Count Plot**: Conditional counts based on operators (e.g., `age >= 18`).
- **Dual Axis**: Allows plotting two disparate variables on a single chart with synchronized X-axis and independent Y-axes (Red = Left, Blue = Right).

Available metrics include agent attributes and simulation-level accumulators:

- `avg_ms_per_tick (sim)`: Real-time simulation performance tracking.
- `k_fertility (sim)`: Macroscopic fertility scaling factor.
- `phi_death_acc / phi_birth_acc (sim)`: Global birth and death rates.

Chapter 6

Full Simulation Tab

The Full Simulation tab is designed for running high-performance, non-interactive simulations.

6.1 Parameters

- **Start/End Year**: Defines the simulation timeline in biological years.
- **Data Save Interval**: Controls how frequently grid data is written to disk.
- **Store dead agents**: When enabled, all deceased agents are archived and written to a NetCDF file alongside the grid data. This uses an asynchronous writer to maintain simulation speed.
- **Output File**: Specifies the path for the simulation recording (.gif) and NetCDF datasets.

Chapter 7

Debugging Fortran Modules

For instructions on how to **create** a new simulation module and register it in the framework, please refer to the *Code Documentation* (Chapter 3: "Developing New Modules"). This chapter assumes that you are using the **test modules** (`mod_test_modules.f95`) for the development of your new modules.

7.1 Development Workflow Overview

Developing and debugging a Fortran module requires switching between the Fortran source code, the build system, and the Python UI. Figure 7.1 illustrates the typical cycle when working with the test modules.

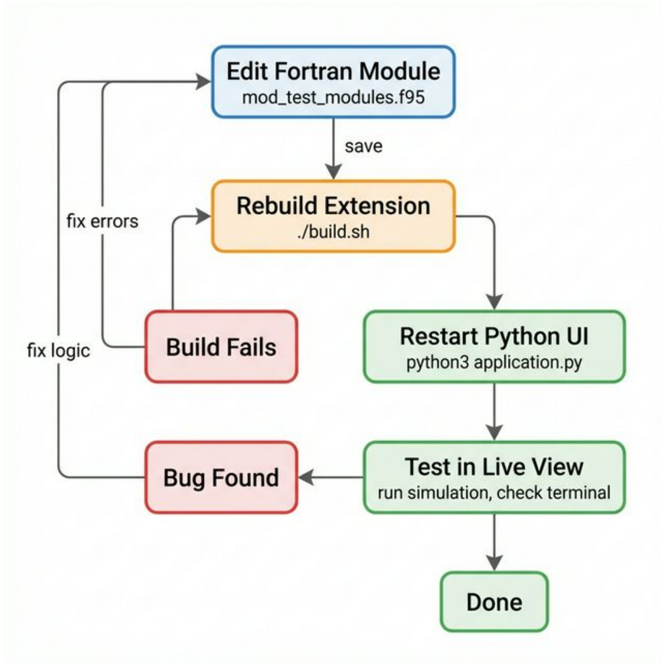

Figure 7.1: The edit–build–test cycle for Fortran module development. After editing Fortran source files, the extension must be rebuilt and the Python UI restarted before changes take effect.

7.2 Print-Based Debugging

Fortran `print` statements are the simplest way to inspect values at runtime. Their output appears in the terminal that launched `application.py`.

```
subroutine my_module(agent_ptr)
    type(Agent), pointer, intent(inout) :: agent_ptr

    print *, "Agent ID:", agent_ptr%id, &
             "pos:", agent_ptr%pos_x, agent_ptr%pos_y
    call flush(6)    ! force output immediately
end subroutine my_module
```

Tip: Always call `flush(6)` after print statements, otherwise output may be lost if the program crashes before the buffer is flushed.

7.3 Common Crash Causes

The most frequent runtime errors when developing Fortran modules are:

Segmentation Fault Usually caused by accessing an unallocated or out-of-bounds array, or dereferencing a null pointer. Common sources:

- Accessing `agents(k, pop)` with `k` beyond `num_humans(pop)`.
- Using grid indices `(gx, gy)` that are outside the grid dimensions.
- Referencing a dead agent (`is_dead = .true.`) that has been compacted away.

Array Bounds Errors Enable runtime bounds-checking by adding `-fcheck=all` to the Fortran compiler flags in the build script. This turns out-of-bounds accesses into clear error messages instead of silent corruption:

```
# In build_fortran.sh, add to FFLAGS:
FFLAGS="$FFLAGS -fcheck=all -g -fbacktrace"
```

`-g` includes debug symbols, and `-fbacktrace` prints a stack trace on crashes.

NaN / Infinity Floating-point issues can propagate silently. Use `-ffpe-trap=invalid` to trap these at the point of origin:

```
FFLAGS="$FFLAGS -ffpe-trap=invalid,zero,overflow"
```

7.4 Configuration Validation

When adding new configurable parameters to `world_config` (see the Code Documentation), verify that:

1. The variable is declared in `mod_config.f95` with a sensible default.
2. The variable is added to the `namelist /config/` block in `mod_read_inputs.f95`.
3. A default value is set **before** the `read(unit, nml=config)` call.
4. The variable is assigned to the `cfg%` struct **after** the read.
5. The variable appears in `basic_config.nml`.

If any step is missing, the parameter may silently retain an uninitialised value.

7.5 Debug Counters Overlay

The Live View provides a "Show Debug Counters" checkbox (in the View Editor tab) that displays an overlay with runtime statistics including death causes, grid-index errors, and movement calls. These counters are updated each simulation tick and are useful for verifying that a module's effects are applied as expected.

To add a custom counter:

1. Add an integer field to the `counter` type in `mod_agent_world.f95`.

2. Increment it inside your module logic.

3. Expose it via `get_debug_stats` in `python_interface.f95`.

7.6 Rebuild & Test Cycle

After making changes to Fortran source files, always:

1. Rebuild the extension:

   ```
   ./build.sh
   ```

2. Check the terminal output for compilation warnings — they often indicate real bugs.

3. Restart `application.py` (the old `.so` is cached by the Python process).

Chapter 8

Python–Fortran Execution Flow

This chapter details the exact mechanics of how the Python frontend orchestrates the lifecycle and execution of the compiled Fortran backend.

8.1 Initialization Sequence

The initialization of the simulation environment is a step-by-step process that prepares memory, loads configurations, generates spatial grids, and populates initial agents. This sequence must run in a specific order:

1. **Setup Config & Paths**: Configures input directories and custom environmental data (HEP) paths.
2. **Active Modules**: Registers which agent/grid subroutines are compiled and active for execution.
3. **Initialization Steps**:
    - `init_sim_step_1`: Initializes internal dimensions and allocations in `mod_agent_world`.
    - `init_sim_step_2_part_1`: Parses the input parameters from the namelist configuration file.
    - `init_sim_step_2_part_2_arrays_only`: Allocates memory space for the spatial grid coordinates.
    - `init_sim_step_2_part_2_chunk`: Populates grid nodes chunk-by-chunk to keep the UI responsive.
    - `init_sim_step_2_part_3`: Loads environmental data from NetCDF datasets.

4. **Spawn Configuration**: Places spawn locations, agents densities, and starting spreads if custom spawn points are set.

5. **Agent Generation**:

   - `init_sim_step_3`: Formally allocates agent records and constructs initial agents.
   - `set_age_distribution_interface` and `init_sim_step_apply_age_dist`: Distributes ages based on specified demographics (optional).

6. **Verification**: `init_sim_step_4` validates the final environment structure and checks configuration parameters.

8.1.1 Interactive Mode (GUI)

In the graphical user interface, this initialization sequence is orchestrated by:

`prepare_simulation()` inside `python/application.py`

This method updates the UI button with a linear progress bar reflecting the status of the steps.

8.1.2 Headless Mode

In non-interactive sweeps or command-line runs, this exact sequence is invoked programmatically by:

`run_simulation_process()` inside `python/headless_simulation.py`

Here, the progress is monitored and printed to standard output or sent to the main process via an IPC queue.

8.2 Simulation Step Execution

Once initialized, the simulation progresses tick-by-tick. Each tick executes the agent behaviors, updates spatial densities, resolves demographics, and tracks metrics.

8.2.1 Interactive Mode (GUI) Loop

When running in Live View mode, the main window runs a Qt timer:

- **Event Handler**: The timeout signal connects to `SimulationWindow.update_sim` in `python/simulation.py`.
- **Execution Call**: This method performs a step loop calling:

  ```
  mod_python_interface.step_simulation(self.t)
  ```

  where `self.t` is the current tick index.
- **Render Update**: Once the step is finished, the method fetches updated statistics and periodically triggers `update_visualization()` to redraw the map.
- **Warmup Ticks**: During the construction of the `SimulationWindow`, `step_simulation(0)` and `step_simulation(1)` are invoked as warmup steps to pre-populate demographics history arrays.
- **Manual Stepping**: In step-by-step or paused mode, manual stepping is triggered via `SimulationWindow.manual_step()`, which calls `step_simulation(self.t)` directly for a set number of ticks.

8.2.2 Headless Mode Loop

In headless execution, a simple CPU-bound loop drives the simulation without UI overhead:

- **Location**: Inside `run_simulation_process()` in `python/headless_simulation`
- **Execution Call**:

  ```
  mpi.step_simulation(t)
  ```

  where `mpi` is the imported bindings object and `t` is the loop iteration index.

Chapter 9

Recent Changes

Chapter 10

Project State Summary - May 5, 2026

10.1 Objective

The primary goals addressed recently were the implementation of **Auto K-Means and DBSCAN**, resolving simulation stability issues, performing a general codebase cleanup, and standardizing the **Birth-Death simulation modules** for consistent scientific modeling.

10.2 Completed Work

10.2.1 1. Dead Agent Archive & NetCDF Storage (April 21st)

- **Objective**: Implement a performance-conscious system for archiving deceased agents for historical analysis.
- **Work Done**:
  - **Fortran Backend**: Added `store_dead_agents` flag to `world_container` and logic to archive agent data before deletion.
  - **Asynchronous Pipeline**: Built an async Python writer using `multiprocessing.Queue` to persist data to NetCDF without blocking the simulation thread.
  - **UI Integration**: Added a checkbox in the "Full Simulation" tab and status indicators for the writer queue.
  - **Bug Fixes**: Resolved segfaults in the F2PY interface by refactoring agent extraction into a clean dataclass-based pipeline.

10.2.2 2. New Module Pattern & World Data Access (April 23rd)

- **Objective**: Standardize how new agent-logic modules are added and how they interact with global state.
- **Work Done**:
  - **Accumulator Reset**: Established a mechanism for `t_tick_accumulators` that automatically reset at the end of each simulation tick.
  - **Module Registration**: Defined a clear 3-step pattern for adding modules (Fortran ID -> Python Interface -> SpawnPointEditor registration) following the `reviewed_agent_motion` template.
  - **World Access**: Identified and documented accessible agent variables via the `world` object to facilitate custom agent-logic development.

10.2.3 3. DBSCAN, Convex Hull & Performance Metrics (April 28th)

- **Objective**: Expand clustering options, improve cluster integrity, and track simulation efficiency.
- **Work Done**:
  - **DBSCAN Implementation**: Added point-based and grid-based DBSCAN clustering in Fortran.
  - **UI Controls**: Added dynamic control over DBSCAN parameters (`eps`, `minPts`) directly from the Python GUI.
  - **Convex Hull Fill**: Integrated a convex hull post-processing step into the grid-based voting mechanism to eliminate holes and noise gaps within identified clusters.
  - **Performance Monitoring**: Implemented `avg_ms_per_tick` tracking. The average time per tick is now displayed in the window title and is available as a plottable variable in the time-series UI.

10.2.4 4. Unified Density Smoothing & Global Metrics (April 29th)

- **Objective**: Standardize density smoothing across all clustering modes and enable real-time plotting of global states.

- **Work Done**:
  - **Unified Smoothing**: Replaced redundant local smoothing in Auto K-Means with a global `human_density_smoothing_radius` parameter.
  - **Iterative Smoothing**: Added `human_density_smoothing_iterations` to allow for aggressive, multi-pass global smoothing.
  - **Global Metrics**: Exported `t_dynamic_state` and `t_tick_accumulators` to Python. Added variables like `k_fertility (sim)`, `phi_death_acc (sim)`, and various `death_* (sim)` debug counters to the UI plotting engine.
  - **Dual-Axis Fix**: Resolved a PyQtGraph closure bug where secondary axes caused the plot canvas to collapse.

10.2.5 5. Birth-Death Refactoring & Logic Consolidation (May 4th)

- **Objective**: Standardize parameter naming and consolidate biological logic for easier scientific review.
- **Work Done**:
  - **Parameter Schema Migration**: Replaced legacy `b1-b4` identifiers with standardized ecological parameters: `r` (growth rate), `NC` (carrying capacity), `Kmin`, and `Kmax`.
  - **Module Consolidation**: Migrated birth/death logic from the obsolete `mod_birth_death_new.f95` into the reviewed and consolidated `mod_reviewed_modules.f95`.
  - **Build System Sync**: Updated the dependency graph and Python interface to support the new parameter schema and modular structure.

10.2.6 6. Architectural Refinement & Critical Fixes (April - May)

- **K-Means Farthest-First Traversal**: Replaced the peak-value search in K-Means with a **Farthest First Traversal (K-means++)** subroutine to ensure geographically remote populations are isolated uniformly.

- **Cluster ID 1-Indexing**: Fixed a visualization bug where Cluster ID 0 was being masked as noise; the system now uses 1-based indexing for all valid clusters.
- **Rendering & Matrix Edge Geometry**: Resolved line rendering issues by translating cluster maps directly through NumPy boolean slicing filters and "burning" 1-pixel white boundaries into the RGBA output matrices.
- **3D Visualization**: Replaced unstable OpenGL 3D views with Matplotlib 3D surfaces in the `compare_smoothing.py` tool for robust cluster verification in headless environments.
- **F2PY Namespace Hotfix**: Fixed a critical bug where localized imports of `mod_python_interface` shadowed the extension module, causing `AttributeError` crashes in the update loop.

10.3 TODO later

- **[LATEX/DOCS]**: Add an entry explaining the Farthest-First Traversal algorithm.
- **[PYTHON/DOCS]**: Document that visual rendering loops for 2D and 3D are fundamentally separate.
- **[PYTHON/DOCS]**: Mark that the backend expects exactly 4-channel RGBA arrays for map updates.

10.3.1 7. Cluster Population Control & Multi-Population Graphing (May 13th)

- **Objective**: Resolve population convergence discrepancies between global and cluster-based modules and improve the Python graphing capabilities.
- **Work Done**:
  - **Divergence Analysis**: Identified that the cluster-based module (`new_birth`) was severely under-suppressing births compared to the global (`reviewed_birth`) module. This was caused by the cluster module relying on the per-cluster accumulator `n_alive_acc(jp)`, which structurally misses agents not assigned to any cluster (e.g. `c_idx = 0`), resulting in massive population under-estimation.

- **Debug Tooling**: Implemented a global-vs-cluster diagnostic summation script within `python_interface.f95` to track the exact magnitude of unassigned agents escaping the control logic.
- **Dynamic UI Plotting via Population Filters**:
- Enhanced `mod_python_interface.get_dynamic_state_stats` and `get_cluster_info` to optionally accept and return stats based on a specific `jp` population index.
- Added a dedicated `pop:` spinbox to the Python GUI (in `application.py`) that feeds into the graph configuration.
- Implemented **"Dominant Population" (`pop = -1`) logic**, allowing graphs to dynamically query the backend tick-by-tick and automatically select and plot statistics (`NC`, `n_agents`, `k_fertility`) specifically for whichever population currently has the most agents within the targeted cluster or globally.

10.3.2 8. Performance Metrics HUD & Population-Specific Mating (May 26th)

- **Objective**: Add real-time wall-clock profiling to identify simulation bottlenecks, and introduce optional reproductive isolation between populations.
- **Work Done**:
  - **Fortran Instrumentation**: Wrapped each phase of `step_simulation` in `python_interface.f95` with `system_clock` calls to measure wall-clock time for permanent modules, active modules, compaction, grid/density, and clustering.
  - **API**: Exposed `set_performance_timing_enabled` and `get_performance_s` to Python via `mod_python_interface`. Implemented as optional with **zero overhead** when disabled.
  - **GUI HUD**: Added a "Show Performance Metrics" checkbox in the "View Editor" tab and a glassmorphic tech-cyan overlay in the simulation window displaying per-phase timing breakdowns.
  - **Population-Specific Mating (`allow_across_populations`)**: Added a new config flag `allow_across_populations` (default `.true.`). When set to `.false.`, the `get_male_from_cell` function is called with `only_population=jp`, enforcing strict reproductive isolation so that females can only mate with males of the same population.

- **Files Modified**: python_interface.f95, mod_agent_world.f95, mod_reviewed_modules.f95, mod_birth_death_new.f95, mod_config.f95, mod_read_inputs.f95, basic_config.nml, config_sandesh.nml, simulation.py, application.py.

10.3.3 9. Critical Bugfix: Hashmap Population Corruption (May 26th)

- **Objective**: Fix a bug causing phantom agents of incorrect populations (e.g. Pop 1 red dots) to appear in single-population simulations.
- **Root Cause**: The remove() subroutine in mod_hashmap.f95 used **open-address linear probing**. When an entry was deleted, subsequent entries in the probe chain were rehashed to fill the hole. The rehash code correctly saved and restored the key and value (array index) fields, but **failed to save or restore the population field**. The moved entry inherited a stale population value from the vacated bucket.
- **Impact**: When get_agent(id, world) later looked up a rehashed agent via get_index_and_pop, it received an incorrect population index. This caused an **out-of-bounds array access** into agents(index, stale_population), reading garbage memory that could appear as an agent with population = 1, rendered as a red dot in the visualization.
- **Fix Applied** (4 changes to mod_hashmap.f95):
  - Added temp_population variable to the rehash loop.
  - Clear population = -1 when initially removing an entry's bucket.
  - Save temp_population alongside temp_key/temp_value when reading entries to move.
  - Write temp_population to the destination bucket and clear population = -1 on the source bucket after moving.
- **File Modified**: src/data_structures/mod_hashmap.f95.

10.3.4 10. Efficient Cluster-Restricted Density Updates (May 26th)

- **Objective**: Speed up the main bottleneck of the simulation step (the $O(N)$ grid sweeps for raw density, combined agent flows, box smoothing, and carrying capacity copies) by restricting updates to cells that belong to active clusters.

- **Work Done**:
  - **New Subroutine** `update_density_and_hep_grid_efficient`: Implemented a cluster-restricted variant of the full density update in `mod_technical_modules.f95`. Instead of sweeping all $n_x \times n_y$ cells, it iterates only over cells belonging to active clusters (`cluster_store%clusters(k)%cell_gx/gy`), computing raw density, agent flows, and HEP copies exclusively for those cells.
  - **Hybrid Scheduling in** `step_simulation`: Modified the density update dispatch in `python_interface.f95` to choose between the full and efficient variants based on tick alignment. On clustering ticks (`mod(t, update_interval) == 0`), the legacy `update_density_and_hep_grid` runs to ensure the full grid is populated before re-clustering. On intermediate ticks, the efficient variant is used.
  - **Boundary Orphan Reset**: Designed and implemented a `was_clustered(:` boundary logical grid array inside the `Grid` type. On re-clustering ticks (`mod(t, update_interval) == 0`), any cell that transitioned OUT of a cluster is immediately zeroed out.
  - **O(1) Neighborhood box smoothing**: Restricted the smoothed density box-filter calculations to cluster cells, looking up neighbor cells directly (which naturally evaluate to `0.0` outside clusters).
  - **O(1) HEP data copies**: Restricted carrying capacity copies strictly to cells within active clusters.
  - **Conditional Startup Fallback**: Embedded conditional fallback inside `step_simulation` in `python_interface.f95` to automatically pivot to the legacy `update_density_and_hep_grid` on tick 0 or when no clusters exist, ensuring robust carrying capacity (`NC`) initialization and preventing agent die-offs.
  - **Config Flag**: Gated behind `efficient_density_updates` (boolean) in the namelist. Currently set to `.false.` in both `basic_config.nml` and `config_sandesh.nml` pending validation that it does not affect simulation accuracy.

- **Status**: Implemented and compiles, but **currently disabled** (`efficient_densit` `= .false.`).

- **Files Modified**: `python_interface.f95`, `mod_technical_modules.f95`, `mod_grid_id.f95`, `mod_config.f95`, `mod_read_inputs.f95`, `basic_config.nml` `config_sandesh.nml`.

10.3.5 11. Responsive Rolling Average for Performance HUD (May 26th)

- **Objective**: Upgrade the performance metrics HUD from a global cumulative average to a responsive rolling average that forgets the distant past and immediately reacts to mid-run toggle switches.
- **Work Done**:
  - **20-Tick Sliding Window Buffer**: Designed and implemented a module-level sliding circular buffer array `perf_history(6, 20)` in `python_interface.f95` that records the exact timings of the last 20 ticks.
  - **Reset on Toggle**: Programmed the rolling timing history array to completely clear and restart fresh whenever performance timing is enabled or disabled in `set_performance_timing_enabled`.
  - **Arithmetic Average stats**: Updated `get_performance_stats` to calculate the exact, unweighted arithmetic average of the recorded window (up to 20 samples), returning clean rolling averages without any clearing side-effects.
- **Files Modified**: `src/interfaces/python_interface.f95`.

10.3.6 12. HEP Data Chunked Loading (May 26th)

- **Objective**: Reduce memory footprint for large HEP NetCDF files ($\approx$ 900 MB) by loading only a window of time slices instead of the entire dataset.
- **Work Done**:
  - **Dynamic Chunk Sizing**: `read_hep_data` in `mod_read_inputs.f95` now queries the HEP file size and calculates a `slices_per_chunk` count targeting $\approx$ 10 MB chunks. Only the first chunk is loaded at startup.
  - **On-Demand Re-Loading**: Added `load_hep_chunk_from_file` subroutine. When `t_hep` advances past the current chunk's range

(chunk_start_t..chunk_end_t), a new chunk is loaded from the NetCDF file with the correct start offset.

  - **Local Index Translation**: update_density_and_hep_grid translates the global t_hep to a local chunk index via local_idx = t_hep - chunk_start_t + 1 before reading grid%hep(:,:,jp,local_idx
  - **Grid Type Extension**: Added slices_per_chunk, chunk_start_t, chunk_end_t metadata fields to the Grid type in mod_grid_id.f95, and a config pointer so the chunk loader can access file paths.

- **Files Modified**: mod_read_inputs.f95, mod_grid_id.f95, mod_agent_world.f mod_technical_modules.f95.

10.3.7 13. Clustering Fix: "Clusters Way Too Small" (May 26th)

- **Objective**: Diagnose and fix the issue where watershed clusters were spatially too tight, capturing only $\approx 55\%$ of alive agents and missing $\approx 45\%$ in peripheral cells.

- **Root Cause**: Three interacting factors:

  1. watershed_threshold was changed from 0.00 → 0.05: With the old value, every non-zero-density cell was included. The new value excluded peripheral cells where smoothed density fell below 0.05.
  2. human_density_smoothing_radius was changed from 3 → 5: The wider kernel spreads density further, reducing peak values and causing more edge cells to drop below threshold.
  3. Redundant internal smoothing in watershed_cluster(): The watershed algorithm was applying 4 additional iterations of box-filter smoothing (radius=4) on top of the already-smoothed human_density_s surface.

- **Fix Applied**:

  - Lowered watershed_threshold to 0.01 in both basic_config.nml and config_sandesh.nml.
  - Removed the redundant 4-iteration internal smoothing loop from watershed_cluster() in mod_watershed.f95.

- **Files Modified**: mod_watershed.f95, basic_config.nml, config_sandesh.nml

10.3.8 14. Bugfix: Out-of-Bounds Array Access in Cluster Accumulators (May 26th)

- **Objective**: Fix memory corruption caused by accessing `cell_cluster_idx` with invalid grid coordinates.
- **Root Cause**: Agents have a default `gx = -1` before being placed on the grid. The accumulator code in `update_agent_age` (`mod_technical_modules.` accessed `cell_cluster_idx(agent%gx, agent%gy)` without bounds-checking. With `gx = -1`, this is an **out-of-bounds Fortran array access** that reads arbitrary memory.
- **Same bug in debug code**: The detailed debug breakup prints in `python_interface.f95` had the identical issue.
- **Fix Applied**:
  - Added `gx >= 1 .and. gx <= nx .and. gy >= 1 .and. gy <= ny` bounds check in `update_agent_age` before accessing `cell_cluster_ic`
  - Added `c_idx <= n_clusters` upper-bound check before using `c_idx` as an index into `clusters(:)`.
  - Added matching bounds checks in the debug breakup code in `python_interface.f95`.
- **Files Modified**: `mod_technical_modules.f95`, `python_interface.f95`.

10.4 Tripwires & Lessons Learned

- **1. Fortran Array Out-of-Bounds is Silent**: Fortran does **not** bounds-check array accesses by default. Accessing `array(-1, 5)` doesn't crash — it silently reads garbage from adjacent memory. This garbage then propagates through the simulation as valid data.
- **2. Config Changes Can Have Non-Obvious Algorithmic Interactions**: The `watershed_threshold` increase from `0.00` to `0.05` seemed harmless — but it interacted catastrophically with the independently-added 4x internal smoothing in the watershed algorithm.
- **3. "Parity" Code Can Be Counter-Productive**: The internal smoothing loop was added to bring watershed clustering into "parity" with Auto K-Means. But the two algorithms have fundamentally different sensitivities.

- **4. Misleading Debug Output Can Send You in Circles**: The debug diagnostic showing "316 agents with invalid/unallocated indices" looked like a clustering algorithm failure. In reality, it was the debug code itself having an out-of-bounds bug.

- **5. `efficient_density_updates = .false.` Made Fix Attempts Into No-Ops**: The earlier fix attempt (hybrid density scheduling, `was_clustered` grid cleanup) was architecturally sound but was entirely guarded by `efficient_density_updates`, which was `.false.` in both configs.